

Ghi chép bài giảng: Stanford CS231n (Spring 2025)

Lecture 8: Attention and Transformers

Ho Hong Phuc Nguyen

Ngày 10 tháng 3 năm 2026

Mục lục

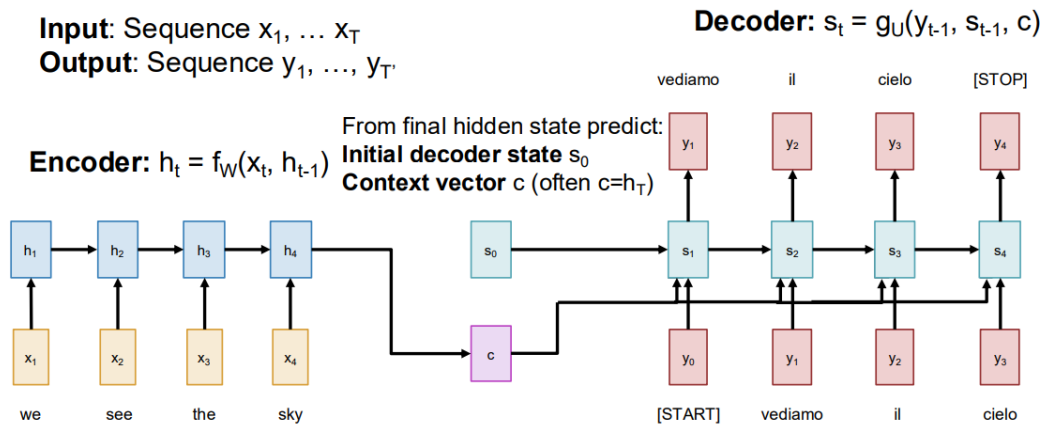
1	Recurrent Neural Networks (RNN) & Bài toán Dịch máy (Translation)	3
1.1	Kiến trúc Encoder - Decoder cơ bản	3
1.2	Vấn đề Bottleneck (Nút thắt cổ chai)	3
2	Cơ chế Attention trong RNN	3
2.1	Quy trình hoạt động (Step-by-step)	3
2.2	Đặc điểm quan trọng	4
3	Tổng quát hóa Attention: Tách rời khỏi RNN	5
3.1	Khái niệm Query, Key, Value	5
3.2	Scaled Dot-Product Attention	5
3.3	Cơ chế Attention với Multiple Query Vectors	6
4	Các biến thể của Attention	6
4.1	Cross-Attention	6
4.2	Self-Attention	6
4.3	Masked Self-Attention	7
4.4	Multi-Head Attention	8
5	Kiến trúc Transformer	9
5.1	Transformer Block	9
5.2	Tại sao Transformer lại thống trị Deep Learning?	10

1 Recurrent Neural Networks (RNN) & Bài toán Dịch máy (Translation)

Để hiểu nguồn gốc của Attention, ta cần nhìn lại cách RNN giải quyết bài toán Sequence-to-Sequence (Seq2Seq), ví dụ như dịch máy từ tiếng Anh sang tiếng Ý.

1.1 Kiến trúc Encoder - Decoder cơ bản

- **Encoder RNN:** Xử lý từng từ của chuỗi đầu vào (tiếng Anh). Nhiệm vụ của nó là tóm tắt toàn bộ nội dung của câu đầu vào thành một vector duy nhất gọi là **Context Vector** (thường là hidden state cuối cùng của mạng Encoder).
- **Decoder RNN:** Sử dụng Context Vector này làm trạng thái khởi tạo (s_0) hoặc đầu vào để sinh ra chuỗi đích (tiếng Ý) từng từ một.



1.2 Vấn đề Bottleneck (Nút thắt cổ chai)

- Sự giao tiếp duy nhất giữa Encoder và Decoder là thông qua Context Vector (kích thước cố định, vd: 1024 floats).
- **Hạn chế:** Việc tóm tắt một câu ngắn thì dễ, nhưng nếu đầu vào là một đoạn văn dài, một cuốn sách hay một văn bản phức tạp, việc nhồi nhét tất cả thông tin vào một vector cố định sẽ gây ra mất mát thông tin nghiêm trọng. Mạng không thể "nhớ" hết mọi thứ.

2 Cơ chế Attention trong RNN

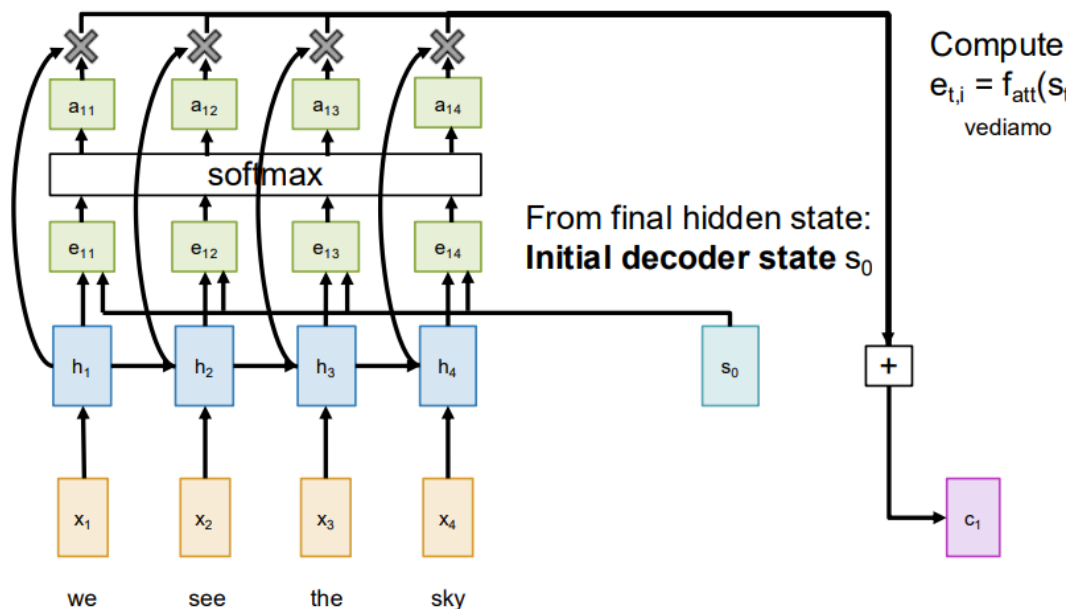
Để giải quyết vấn đề Bottleneck, ý tưởng cốt lõi của Attention là: **Không ép mạng phải tóm tắt tất cả vào một vector cố định nữa. Thay vào đó, cho phép Decoder "nhìn lại" (look back) toàn bộ chuỗi đầu vào tại MỖI bước thời gian nó sinh ra từ mới.**

2.1 Quy trình hoạt động (Step-by-step)

Tại mỗi bước sinh từ của Decoder (với hidden state hiện tại là s_t):

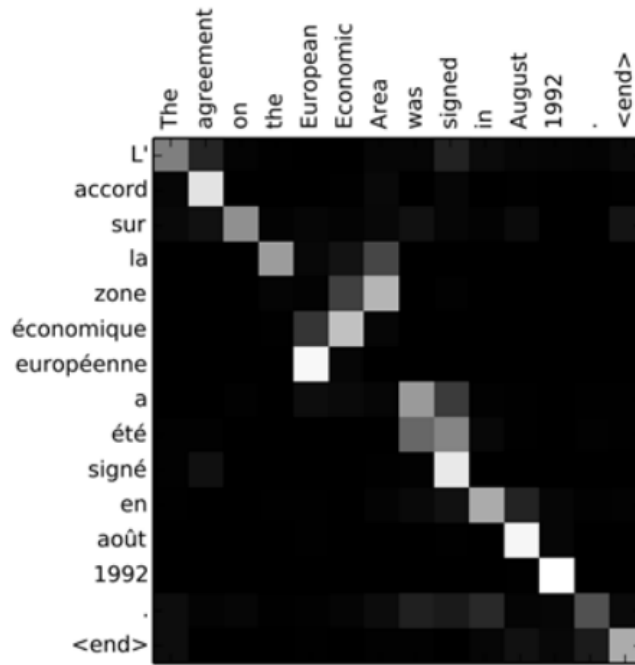
1. **Tính Alignment Scores (Điểm liên kết):** So sánh hidden state s_t của Decoder với TẤT CẢ các hidden states của Encoder ($h_1, h_2, \dots, h_N$) thông qua một hàm tuyến tính f_{att} . Kết quả là một tập hợp các giá trị vô hướng.
2. **Tính Attention Weights (Trọng số Attention):** Đưa các Alignment Scores qua hàm softmax để đưa chúng về dạng phân phối xác suất (tổng bằng 1, các giá trị trong khoảng $[0, 1]$).
3. **Tính Context Vector mới (c_t):** Tính tổng có trọng số (linear combination) của tất cả các hidden states của Encoder, với trọng số chính là các Attention Weights vừa tính.

4. **Dự đoán:** Sử dụng c_t kết hợp với s_t để dự đoán từ tiếp theo.



2.2 Đặc điểm quan trọng

- **Cơ chế tự học (Self-learning Attention):** Decoder tự học cách tập trung (attend) vào các phần quan trọng của câu đầu vào đối với từ đang cần dịch mà **không cần gán nhãn thủ công (no direct supervision)**. Quá trình này hoàn toàn khả vi (differentiable) và được học thông qua *gradient descent*.
- **Khả năng giải thích (Interpretability):** Ma trận trọng số Attention giúp ta hình dung mạng đang "nhìn" vào đâu. Ví dụ:
 - *Cấu trúc đường chéo (diagonal):* Thể hiện sự tương ứng 1-1 giữa các từ.
 - *Cấu trúc chéo ngược:* Thể hiện sự khác biệt về cấu trúc ngữ pháp giữa hai ngôn ngữ.
- **Khởi tạo trạng thái ẩn (Decoder Hidden State Initialization):** Có hai phương pháp phổ biến để thiết lập trạng thái ban đầu cho Decoder:
 - *Cách 1:* Sử dụng trạng thái ẩn cuối cùng của Encoder để khởi tạo cho Decoder (giúp chuyển tiếp ngữ cảnh nên từ câu nguồn).
 - *Cách 2:* Khởi tạo tất cả các giá trị trong trạng thái ẩn bằng 0 (Decoder sẽ dựa hoàn toàn vào cơ chế Attention và vector ngữ cảnh để bắt đầu).



Hình 1: Enter Caption

3 Tổng quát hóa Attention: Tách rời khỏi RNN

Cơ chế Attention thực chất là một phép toán cực kỳ mạnh mẽ và có thể đứng độc lập như một Neural Network Layer riêng biệt mà không cần đến RNN.

3.1 Khái niệm Query, Key, Value

Giống như cơ chế tìm kiếm (Search Engine):

- **Query (Q):** Điều bạn đang muốn tìm (Tương đương hidden state của Decoder).
- **Key (K):** Các nhãn/từ khóa của dữ liệu để đối chiếu (Tương đương hidden state của Encoder dùng để tính điểm so khớp).
- **Value (V):** Nội dung thực sự bạn muốn lấy ra (Tương đương hidden state của Encoder dùng để cộng gộp tạo Context Vector).

Mạng sẽ tự động học các ma trận trọng số W_Q, W_K, W_V để chiếu (project) dữ liệu đầu vào thành các vector Q, K, V tương ứng.

3.2 Scaled Dot-Product Attention

Thay vì dùng một mạng neural nhỏ (f_{att}) để tính độ tương đồng, ta dùng phép nhân vô hướng (Dot Product) cho đơn giản và hiệu quả.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Tại sao phải chia cho $\sqrt{d_k}$? Khi số chiều của vector (d_k) lớn, giá trị của phép nhân vô hướng có thể rất lớn, đẩy các giá trị hàm softmax vào vùng bị bão hòa. Điều này gây ra hiện tượng **Vanishing Gradients**. Việc chia cho $\sqrt{d_k}$ giúp ổn định phân phối và cải thiện gradient flow.

3.3 Cơ chế Attention với Multiple Query Vectors

Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

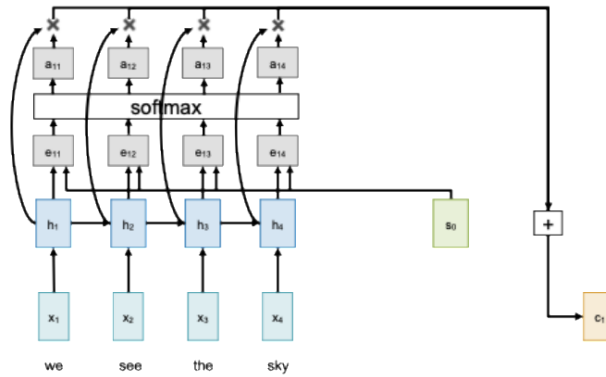
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Changes

- Use scaled dot product for similarity
- Multiple **query** vectors
- Separate **key** and **value**

4 Các biến thể của Attention

4.1 Cross-Attention

Queries đến từ một nguồn (vd: Decoder), trong khi Keys và Values đến từ một nguồn khác (vd: Encoder). Dùng trong dịch máy hoặc Image Captioning (Q từ text, K và V từ ảnh).

Cross-Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Each **query** produces one **output**, which is a mix of information in the **data** vectors

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

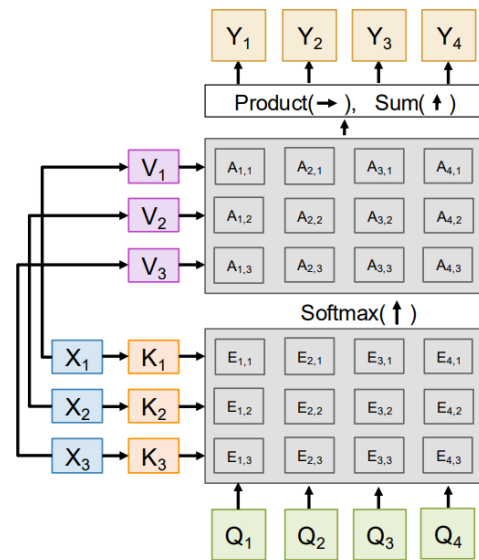
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



4.2 Self-Attention

- Mọi thành phần Q, K, V đều được nội suy (project) từ **cùng một chuỗi đầu vào**. Mỗi vector trong chuỗi tự so sánh nó với tất cả các vector khác trong cùng chuỗi đó.
- **Permutation Equivariance**: Self-Attention xem đầu vào như một "tập hợp"(set) thay vì "chuỗi"(sequence). Nếu xáo trộn thứ tự đầu vào, đầu ra cũng xáo trộn tương ứng, bản chất tính toán không đổi.
- **Positional Embeddings**: Vì Self-Attention không tự hiểu được thứ tự, ta phải cộng thêm thông tin vị trí (positional embeddings) vào các vector đầu vào để mạng phân biệt được từ nào đứng trước, từ nào đứng sau.

Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Each **input** produces one **output**, which is a mix of information from all **inputs**

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

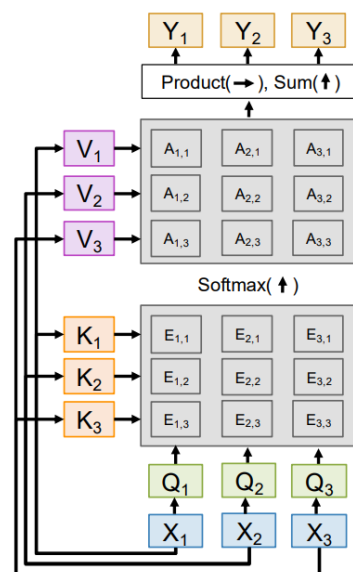
$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$

Shapes get a little simpler:
- N input vectors, each D_{in}
- Almost always $D_Q = D_V = D_{out}$



4.3 Masked Self-Attention

Sử dụng trong Language Modeling (dự đoán từ tiếp theo). Ta gán điểm số Alignment của các từ trong tương lai thành $-\infty$ trước khi tính Softmax. Hậu quả là Softmax sẽ biến các vị trí này thành 0, ngăn không cho mô hình "nhìn trộm" tương lai.

Masked Self-Attention Layer

Don't let vectors "look ahead" in the sequence

Inputs:

Input vectors: $\mathbf{X}$ [$N \times D_{in}$]

Key matrix: $\mathbf{W}_K$ [$D_{in} \times D_{out}$]

Value matrix: $\mathbf{W}_V$ [$D_{in} \times D_{out}$]

Query matrix: $\mathbf{W}_Q$ [$D_{in} \times D_{out}$]

Override similarities with $-\infty$; this controls which inputs each vector is allowed to look at.

Computation:

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times D_{out}$]

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times D_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times D_{out}$]

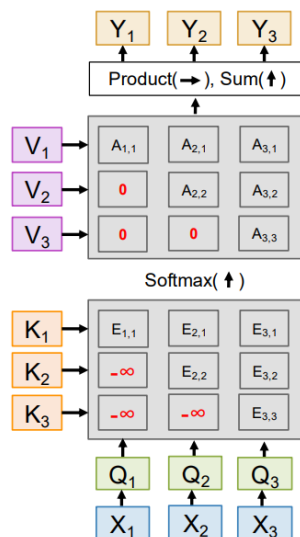
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N \times N$]

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N \times N$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times D_{out}$]

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



4.4 Multi-Head Attention

Multiheaded Self-Attention Layer

Run H copies of Self-Attention in parallel

Inputs:

Input vectors: $X [N \times D]$

Key matrix: $W_K [D \times HD_H]$

Value matrix: $W_V [D \times HD_H]$

Query matrix: $W_Q [D \times HD_H]$

Output matrix: $W_O [HD_H \times D]$

In practice, compute all H heads in parallel using batched matrix multiply operations.

Computation:

Queries: $Q = XW_Q [H \times N \times D_H]$

Keys: $K = XW_K [H \times N \times D_H]$

Values: $V = XW_V [H \times N \times D_H]$

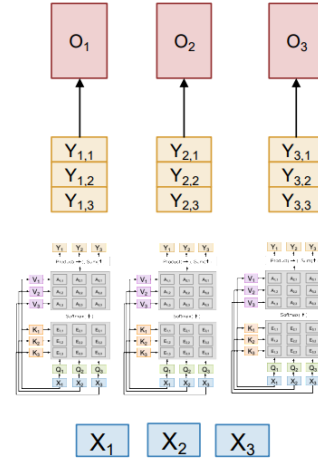
Similarities: $E = QK^T / \sqrt{D_Q} [H \times N \times N]$

Attention weights: $A = \text{softmax}(E, \text{dim}=2) [H \times N \times N]$

Head outputs: $Y = AV [H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $O = YW_O [N \times D]$

Used everywhere in practice.



Hình 2: Cấu trúc Multi-Head Attention với các phép chiếu tuyến tính và ma trận đầu ra.

Thay vì chạy một phép tính Attention duy nhất trên toàn bộ chiều dữ liệu, ta chia nhỏ không gian vector và thực hiện H phiên bản Attention song song (gọi là các "heads").

- **Cơ chế hoạt động:** Mỗi head sử dụng các tập trọng số riêng biệt để chiếu dữ liệu vào các không gian biểu diễn khác nhau. Điều này cho phép mô hình đồng thời tập trung vào các khía cạnh khác nhau của câu (ví dụ: một head tập trung vào ngữ pháp, một head tập trung vào ngữ nghĩa dài hạn).
- **Ma trận đầu ra (W^O):** Sau khi tính toán xong, kết quả từ các head ($head_1, \dots, head_H$) được nối lại (*concatenate*) thành một vector duy nhất. Vector này sau đó được nhân với một ma trận trọng số đầu ra W^O để tổng hợp thông tin từ tất cả các heads và đưa về chiều không gian ban đầu.
- **Tối ưu hóa tính toán (Sự "thông minh" trong triển khai):** Thay vì tính toán H lần riêng biệt, ta gộp các ma trận trọng số của tất cả các heads lại thành 4 ma trận lớn duy nhất:

- $W^Q \in \mathbb{R}^{d_{model} \times d_{model}}$
- $W^K \in \mathbb{R}^{d_{model} \times d_{model}}$
- $W^V \in \mathbb{R}^{d_{model} \times d_{model}}$
- $W^O \in \mathbb{R}^{d_{model} \times d_{model}}$

1. QKV Projection

$[N \times D] [D \times 3HD_H] \Rightarrow [N \times 3HD_H]$

Split and reshape to get Q, K, V each of shape $[H \times N \times D_H]$

2. QK Similarity

$[H \times N \times D_H] [H \times D_H \times N] \Rightarrow [H \times N \times N]$

3. V-Weighting

$[H \times N \times N] [H \times N \times D_H] \Rightarrow [H \times N \times D_H]$

Reshape to $[N \times HD_H]$

4. Output Projection

$[N \times HD_H] [HD_H \times D] \Rightarrow [N \times D]$

Việc này chuyển đổi bài toán thành các phép nhân ma trận lớn (*Batch Matrix Multiplications*), giúp tận dụng tối đa khả năng tính toán song song cực mạnh của GPU.

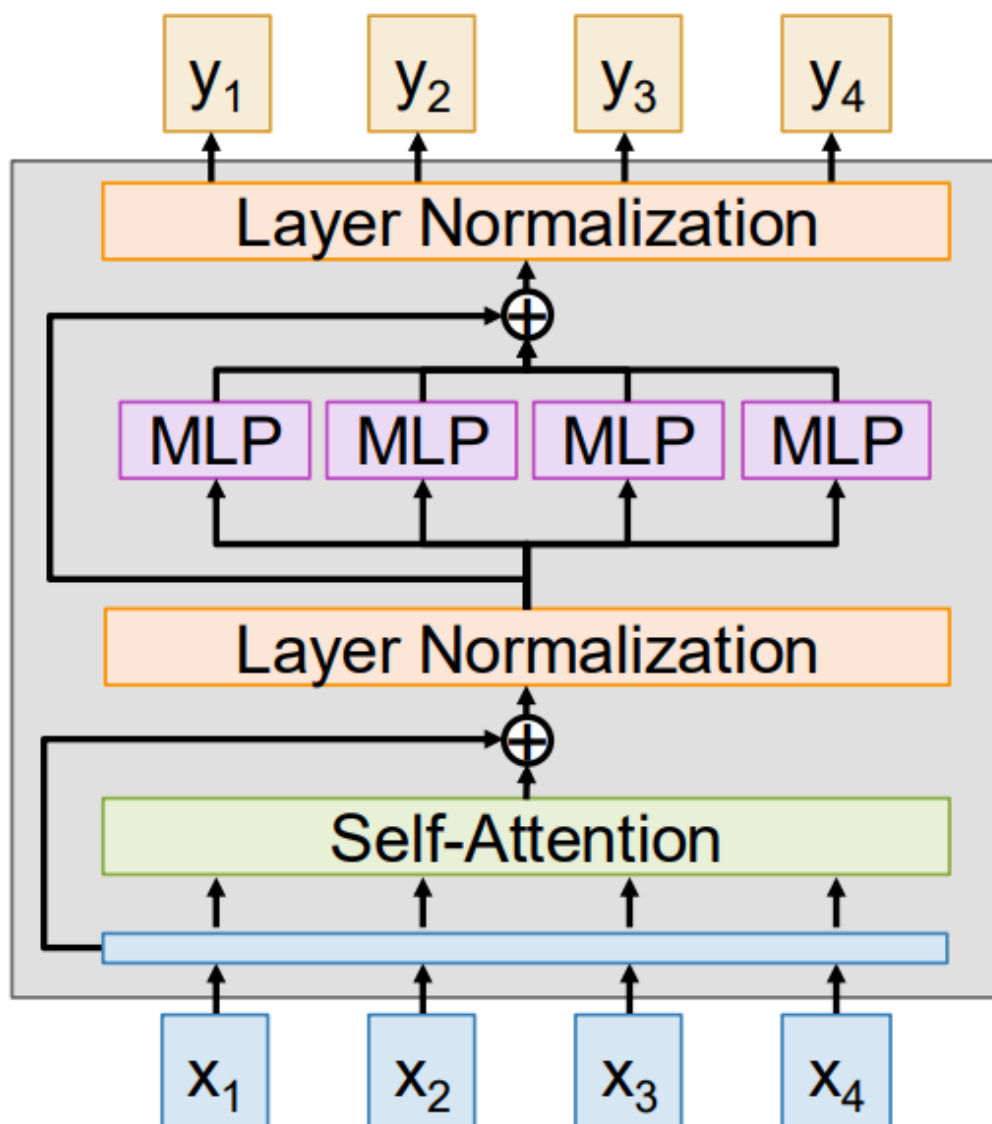
5 Kiến trúc Transformer

Transformer là kiến trúc mạng nơ-ron từ bỏ hoàn toàn RNN/CNN, đặt **Self-Attention** làm hạt nhân chính.

5.1 Transformer Block

Một khối Transformer tiêu chuẩn bao gồm các thành phần:

1. **Multi-Head Self-Attention:** Giúp các token/vector giao tiếp và thu thập thông tin ngữ cảnh từ toàn bộ chuỗi.
2. **Residual Connections & Layer Normalization:** Giúp ổn định quá trình huấn luyện và tránh vanishing gradients cho các mạng sâu.
3. **Feed-Forward Network (MLP):** Một mạng nơ-ron 2 lớp áp dụng **độc lập** lên từng vector vị trí. (Self-Attention lo phần giao tiếp giữa các token, MLP lo phần suy luận riêng cho từng token).



5.2 Tại sao Transformer lại thống trị Deep Learning?

Mô hình	Ưu điểm	Nhược điểm
RNN	Xử lý chuỗi dài vô hạn (lý thuyết).	Tính toán tuần tự (Sequential) → Rất chậm, khó tận dụng sức mạnh GPU. Nút thắt cổ chai thông tin.
CNN	Song song hóa tốt.	Receptive field nhỏ, cần xếp chồng nhiều lớp để nhìn được toàn cảnh → Vẫn mang tính tuần tự phần nào.
Attention / Transformer	Khả năng song song hóa cực cao (chỉ là các ma trận nhân với nhau). Tầm nhìn toàn cục (Mọi token tương tác trực tiếp với nhau trong $O(1)$ bước).	Tính toán bậc 2 ($O(N^2)$) theo độ dài chuỗi (tốn RAM/Compute khi N lớn).

Kết luận: Dù $O(N^2)$ có vẻ đắt đỏ, nhưng vì phần cứng (GPU) phát triển theo hướng tăng cường khả năng xử lý song song thay vì tăng tốc độ lõi đơn, Transformer với khả năng song song hóa tuyệt đối đã vươn lên trở thành kiến trúc State-of-the-Art cho từ xử lý ngôn ngữ tự nhiên (LLMs), hình ảnh (Vision Transformers), cho đến âm thanh.